

SRW3 Phase 1E — Authenticated State and State-Commitment Binding

****Status:** PHASE 1E EXECUTED — research question answered with a qualified, fully-bounded positive result.** This report was produced from a fresh clone of phase1d-r1-complete (closure commit 668a6519b017de226b25a6b1d6bf8736a731970c, tag peeled to the scientific anchor 85f4be5ad5fc525540b17b7a27ea27eb34e700a3) on a dedicated phase1e branch. Phase 1D-R1 was not modified: `git diff over k/` and `python-gen/` against the closure commit is empty, and every Phase 1E extension is additive (the `linCtxP` discipline).

1. Research question and answer

Question. Can the SRW3 commitment gate make its security decision depend on an authenticated state commitment, so that a presented post-state is cryptographically anchored to an authoritative state commitment rather than merely internally self-consistent?

****Answer:** YES at Level 2, under explicitly stated assumptions — with the level separation preserved.** The gate was extended (additively) so that its accept decision requires the presented state to match a fixed-depth binary Merkle root that is (i) recomputed by the verifier from the presented artifacts (Mode S) and (ii) equal to an explicit authoritative anchor (Mode A). Under Keccak256 collision resistance, a record accepted by the extended gate presents a post-state that is BOTH internally self-consistent (the Phase 1D-R1 L7 exact-equality level) AND anchored to the authoritative commitment. The strengthening is real and demonstrated: the anchor layer rejects the L7-consistent hidden-write class (CM-E7) that the Level-1 surface cannot even see, because in the self-contained model the producer presents ALL artifacts and the Level-1 checks alone have no independent reference. What this result is NOT: it does not authenticate historically true execution effects (Level 3, NOT YET MECHANIZED), and it does not establish anchor provenance (REQUIRES CLIENT/PROTOCOL SUPPORT).

2. Starting state and baseline regression

The handoff note mandated re-running the permanent suites before any Phase 1E work, so regressions would be attributable to Phase 1E changes only. The environment was rebuilt from the pinned recipe after the sandbox reset (K v7.1.337, Z3 4.13.3-1, clang/LLVM-15 15.0.6-4+b1, libsecp256k1 0.5.0 with soname shims, KEVM v1.0.921 source, blockchain-k-plugin @ 207ae512; krypto

shim rebuilt with the keccak-multiblock fix). Baseline results

(transcripts/audit/phase1e/part0_baseline_regression.txt):

Suite	Result
L7 permanent adversarial suite	10/10 PASS
Legacy mutation harness (21 classes)	21/21 rejected
Comparison audit	33/33 probes, zero deviations
Authorized-dishonest producer	REJECTED at L7-STATE
K composition suite (fresh LLVM re-kompile)	pos=valid;neg1=neg2=neg3=invalid-state-composition

3. The minimal authenticated-state model (FROZEN)

phase1e/semantics/MODEL.md is the normative model specification. Design points:

- **State and universe.** Keys are flattened (app, slot) pairs ($k = \text{app} \cdot 2^{32} + \text{slot}$). The canonical key universe of a record is $U = \text{sort}(\text{keys}(\text{pre}) \cup \text{keys}(\text{effects}) \cup \text{keys}(\text{post}))$ — determined entirely by the artifacts the record already digests, so the producer gains no universe freedom. Capacity 256 leaves (depth 8); overflow is an explicit model refuse (AUTH-CAPACITY), never a silent truncation. The reserved pad key $\text{PAD} = 2^{64}-1$ fills the tree; a real key equal to PAD is refused (AUTH-PADKEY).
- **Tree.** Fixed-depth binary Merkle tree, 256 leaves, always. Domain-separated encodings: leaf bytes $0x00 \parallel \text{app}_4 \parallel \text{slot}_4 \parallel \text{presence} \parallel \text{value}_4$, node bytes $0x01 \parallel \text{left} \parallel \text{right}$; root = top node hash; a full canonical tree byte string ($0x02 \parallel \text{depth}_4 \parallel |U|_4 \parallel \text{leaf bytes}$) provides an injective representation for certificates. Deterministic representation: one unique byte string and one unique root per (U, post).
- **Proofs.** Content-binding inclusion/non-membership proofs: 260 bytes = leaf index + 8 bottom-up sibling digests. Verification recomputes the leaf from the CLAIMED content and walks the path; acceptance holds iff the recomputed digest equals the root. Update proofs re-derive the old root first, then the new root from the same siblings.
- **Record extension (strictly additive).** $\text{LinRecE} = (\text{LinRec}, \text{stateRoot}, \text{childE})$ with $\text{CanonCoreE} = \text{legacy canonical full bytes} \parallel \text{stateRoot}$ — the legacy canonical bytes are a strict PREFIX of the extended ones, and the legacy 12-layer chain runs unchanged on the legacy projection. The legacy child commitment is therefore unaffected by the extension (version compatibility by construction).
- **Verification modes, explicitly separated.** Mode S: the verifier recomputes the root from the presented artifacts. Mode A: the verifier

additionally requires `stateRoot == anchor`, where the anchor is an explicit context input standing for an independently published authoritative state commitment.

- **This is not an Ethereum MPT.** No trie, no RLP, no Ethereum state-root property is claimed anywhere; the model is a minimal fixed-depth tree over the SRW3 canonical universe, built to be fully executable and mechanizable.

4. Experiments and evidence tiers

Experiment	Statement	Tier
E1 authenticated read	AUTH-READ: honest content-binding proofs verify; acceptance iff the recomputed walk equals the claimed root (no acceptance slack); ill-formed proofs never verify	PROVED BY K (AR1, AR2a, AR2b, AR3, AR4) + Python mirror
E2 authenticated update	AUTH-UPDATE: a verified-basis proof update equals the new-content walk over the same siblings; unverified basis yields the .Bytes sentinel	PROVED BY K (AU1, AU2, AU3) + Python mirror
E3 root-bound lineage	LinRecE binds <code>stateRoot</code> into the lineage; <code>childE = H(CanonCoreE)</code> covers sig + <code>stateRoot</code> ; prefix compatibility	PROVED BY K (RL2, RL4, RL5, RL6)
E4 gate soundness	Acceptance (<code>valid-a</code>) implies the full certificate: <code>legacy chain valid ∧ capacity ∧ padkey ∧ stateRoot == recomputed root ∧ stateRoot == anchor ∧ childE correct</code>	PROVED BY K branch-wise (RL4, RL5, RL6, GA2, GA3a, GA3b — every extension verdict branch characterized exactly); single mega-implication form GA1 BLOCKED (simplification budget), disclosed
E5 authenticated composition	Two-transition chain with root threading accepted; stale-anchor replay across the chain boundary rejected	PROVED BY K per-step (branch-wise set) + DEMONSTRATED (K chain= <code>valid-a</code> ; stale-replay= <code>invalid-anchor</code> ; Python C1)
E6 execution-binding boundary	Tree, proofs, and verifier run over REAL EVM storage with REAL keccak; honest anchor accepted, stale anchor rejected with restore	DEMONSTRATED BY KEVM (evm_auth_commit / evm_auth_stale); anchor provenance REQUIRES CLIENT/PROTOCOL SUPPORT; NOT a Level-3 result

Anti-vacuity. The gate premise is satisfiable: `pos=valid-a` is a permanent demo case on the LLVM backend with genuine keccak + secp256k1 (the honest witness), mirrored by the Python E0 witness and the KEVM commit demo. The vacuity probe NC3 (designed to show "verification never accepts") fails correctly. No theorem relies on an unsatisfiable gate condition.

5. Countermodels (all demonstrated, all rejected)

Each countermodel is a permanent regression case in the Python suite and, where noted, mirrored in the K demo suite on real crypto:

ID	Construction	Rejected at
CM-E1	Forged value in an inclusion proof vs a fixed root	proof verification (E1 semantics)

I D	Construction	Rejected at
C M - E 2	Tampered sibling path (one byte)	proof verification
C M - E 3	Extra key in presented post (the L7 attack, authenticated)	legacy composition layer first (Python L7-STATE / K invalid-state-composition); root diverges independently
C M - E 4	Omitted key from presented post	L7-STATE first; root diverges
C M - E 5	Changed value in one leaf	L7-STATE first; root diverges
C M - E 6	Foreign stateRoot, all crypto genuine, childE consistent	L13-STATEROOT / invalid-state-root
C M - E 7	HIDDEN WRITE outside the declared footprint, L7-consistent (eff' carries the write, post' matches), every digest/signature genuine	L14-ANCHOR / invalid-anchor — the anchor rejects what Level 1 cannot see (Python + K neg3 + symbolic GA2)
C M - E 8	Stale anchor (previous commitment as current)	L14-ANCHOR / invalid-anchor (Python + K neg4)
C M - E 9	Partial-proof attack: valid subset proofs are individually sound, but gate acceptance without the full-universe root is structurally impossible	gate rejects (L7 requires the full map; no partial acceptance path)

Model refuses are demonstrated separately: AUTH-CAPACITY and AUTH-PADKEY (Python REFUSE cases; K GA3a/GA3b verdict characterizations).

6. FINDING 1E-LINAPPLY-MULTISLOT (disclosed defect + additive fix)

While binding the KEVM layer, the Phase 1E work exposed a latent defect in the FROZEN Phase 1D-R1 composition function `LinApplyEff` (`srw3lin-r1.k`): its account-level recursion removes the whole account after applying only the MINIMUM-KEY slot, so effects with multiple slots per account lose every slot except the first (demonstrated: `slots4097=2,1` in the debug transcript).

Consequences and boundaries, stated precisely:

- The Phase 1D-R1 K composition suite did not expose this because its positive case DEFINED the honest post-state as `LinApplyEff(Q, E)` itself — the compared terms coincided (tautology). The negative cases (extra, missing, changed entries) remain valid demonstrations.
- Classification: a COMPLETENESS defect (honest multi-slot records are falsely rejected by the frozen K composition check). It does NOT weaken the 1D-R1 soundness claims: extra/missing/changed entries were and remain rejected, and the permanent L7 suite was governed by the Python verifier, whose `apply_effects` was always correct (per-slot).
- Fix: ADDITIVE in Phase 1E — `LinApplyEff2 + VerifyLineagePE` (composition-aware chain with the corrected `apply`) in `srw3auth.k §0`; the Phase 1E chain uses the corrected base. Frozen modules untouched. The demo now constructs posts INDEPENDENTLY of any `apply` function and demonstrates `applyfix:old=false,new=true,oldmain=true,newmain=true`.

7. Cross-layer byte consistency

`phase1e/transcripts/part6_crosslayer_bytes.txt` (methodology = the 1D-R1 Part-8 precedent): **18/18 byte-equality checks MATCH** across Python ↔ K-abstract (Illum) ↔ KEVM over real EVM storage — canonical tree encodings, Merkle root, stateRoot, childE, legacy child, 260-byte proofs, record digests, evidence, signature, and the anchored verdict itself. The KEVM anchor was computed by the Python reference and embedded in the demo program, so acceptance certifies Python↔KEVM root equality by construction.

8. Assumption census (nothing hidden)

ID	Assumption	Classification		
A E 1	Keccak256 collision resistance (digest - distinctness under differing content)	REQUIRES CRYPTOGRAPHIC ASSUMPTION; the boundary is pinned by designed-stuck negative control NC1		
A E 2	Canonical-encoding injectivity (leaf/node/tree/proof/record)	STRUCTURAL (fixed-width fields, ascending order) — not cryptographic		
A E 3	Anchor authenticity (R* is the authoritative root)	ASSUMED gate input; provenance REQUIRES CLIENT/PROTOCOL SUPPORT		
A E 4	Model capacity: \	U\	≤ 256, keys < 2 ³²	explicit REFUSE on violation

ID	Assumption	Classification		
A-E5	Reserved pad key never used by real state	explicit REFUSE on violation		

9. What is proved, validated, and open

- **What is now proved (PROVED BY K).** The exactness of content-binding proof verification (acceptance $\iff$ recomputed walk equals the root, both directions), proof determinism, verified-basis update semantics, the record/root binding layers, and the branch-wise characterization of every extension verdict of the extended gate (invalid-capacity, invalid-padkey, invalid-state-root, invalid-anchor, invalid-childe, valid-a) — 15 claims PROVED by kprove on the Haskell backend (AR1, AR2a, AR2b, AR3, AR4; AU1, AU2, AU3; RL2, RL4, RL5, RL6; GA2, GA3a, GA3b).
- **What is experimentally validated (DEMONSTRATED).** The full gate/demo suites on the LLVM backend with real crypto (abstract + KEVM), the chain/stale-replay cases, the applyfix demonstration, and the 18/18 cross-layer byte certificates.
- **What was refuted (designed-fail controls).** NC2 (acceptance with a different walked root) and NC3 (vacuity probe) fail as designed; NC1 (hash injectivity) is stuck by design — the honest boundary of A-E1.
- **What remains unproved / open (unchanged or explicitly bounded).** GA1's single mega-implication form is BLOCKED at the simplification budget (its content is carried branch-wise); RL1 and RL3a/RL3b identities are NOT MECHANIZED symbolically (subsumed where it matters; pinned concretely by bytes); Level-3 execution-effect authentication remains NOT YET MECHANIZED; anchor provenance remains REQUIRES CLIENT/PROTOCOL SUPPORT; the result is a concrete demonstration within the modeled verifier domain, not a general cryptographic theorem.

PHASE 1E RESULTS

Research question: can gate security decisions depend on an authenticated state commitment anchoring the presented state to an authoritative commitment?

Status: ANSWERED — qualified positive at Level 2 under stated assumptions; level separation preserved

Authenticated state model: fixed-depth binary Merkle tree (D=8, 256 leaves) over the canonical key universe

$U = \text{sort}(\text{keys}(\text{pre}) \cup \text{keys}(\text{eff}) \cup \text{keys}(\text{post}))$;

domain-separated leaf/node encodings; content-binding 260-byte proofs; NOT an Ethereum MPT

Representation: deterministic + injective by construction (fixed-width fields, ascending order); full tree byte string; additive record extension

LinRecE with prefix-compatible CanonCoreE

Cryptographic assumption(s): A-E1 Keccak256 collision resistance (REQUIRES CRYPTOGRAPHIC ASSUMPTION); A-E3 anchor authenticity (ASSUMED / REQUIRES CLIENT/PROTOCOL SUPPORT); A-E2 injectivity is STRUCTURAL; A-E4/A-E5 explicit model refuses

E1 authenticated read: PROVED BY K (AR1/AR2a/AR2b/AR3/AR4) + Python

E2 authenticated update: PROVED BY K (AU1/AU2/AU3) + Python

E3 root-bound lineage: PROVED BY K (RL2/RL4/RL5/RL6)

E4 gate soundness: PROVED BY K branch-wise (RL4/RL5/RL6/GA2/GA3a/GA3b: every extension verdict branch characterized); GA1 single form BLOCKED (budget, disclosed); non-vacuity witness

pos=valid-a (K/llvm, real krypto) + Python E0

E5 authenticated composition: PROVED BY K per-step + DEMONSTRATED (chain=valid-a; stale-replay=invalid-anchor)

E6 execution-binding boundary: DEMONSTRATED BY KEVM over real storages with real keccak; NOT Level-3; anchor provenance

REQUIRES CLIENT/PROTOCOL SUPPORT

Countermodels: all demonstrated and rejected (Section 5):

CM-E1: forged value in proof -> proof verification rejects (Python+K)

CM-E2: tampered sibling path -> proof verification rejects (Python+K)

CM-E3: extra key (L7 attack, authenticated) -> legacy composition first, root diverges (Python+K)

CM-E4: omitted key -> L7-STATE first, root diverges (Python+K)

CM-E5: changed value -> L7-STATE first, root diverges (Python+K)

CM-E6: foreign stateRoot, genuine crypto -> invalid-state-root (Python+K)

CM-E7: L7-consistent hidden write, genuine crypto -> invalid-anchor (Python+K+symbolic GA2)

CM-E8: stale anchor -> invalid-anchor (Python+K+KEVM stale demo)

CM-E9: partial-proof claim -> gate structurally requires full-universe root; rejects (Python+K)

Python/K consistency: 5/5 Phase 1E byte checks (root, stateRoot, childE, legacy child, 260B proof) + permanent suites (24/24 adversarial, 12/12 mutations)

K/KEVM consistency: 18/18 three-layer byte checks MATCH; KEVM demos: honest anchor COMMIT (valid-a), stale anchor REJECT+restore (invalid-anchor)

Hidden-effect boundary: unchanged and explicit: presented-effect consistency + Level-2 anchoring ONLY; no historically-true-execution claim

Ethereum/MPT status: NOT an Ethereum MPT; no trie/MPT property claimed; minimal model per MODEL.md

What is now proved: proof-verification exactness (iff), update semantics, record/root binding, branch-wise gate certificate — 15 kprove claims PROVED

What is only experimentally LLVM gate/demo suites with real krypto validated: (abstract + KEVM), chain/stale-replay, applyfix, 18/18 cross-layer byte certificates

What remains unproved: GA1 single-implication form (BLOCKED, budget; content carried branch-wise); RL1/RL3a/RL3b symbolic identities (NOT MECHANIZED; subsumed + pinned concretely); Level-3 execution authentication (NOT YET MECHANIZED); anchor provenance (REQUIRES CLIENT/PROTOCOL SUPPORT)

What was refuted: NC2 (acceptance slack) and NC3 (vacuity) fail as designed; the old-apply composition on a new-slot write demonstrated false (applyfix:old=false) — finding fixed additively

Artifact SHA-256: see SRW3-Phase1E-Artifact-Bundle.zip.sha256 and phasele/provenance/ (MANIFEST, hashes)

Git branch: phasele (derived from phaselD-r1-complete closure commit 668a6519...; no history rewrite)

Git commit: recorded in phasele/provenance/ after final commit (see PROVENANCE block)

Phase 1E completion verdict: COMPLETE — all completion criteria evaluated below; no criterion silently waived

10. Completion criteria evaluation

#	Criterion	Verdict
1	Minimal authenticated-state model designed, deterministic, fully executable, suitable for K mechanization	SATISFIED (MODEL.md frozen; executable in Python + K hs/llvm + KEVM)
2	Root uniqueness under explicit cryptographic assumptions	SATISFIED (A-E1/A-E2 stated; injectivity structural; refuses explicit)
3	Explicit inclusion/non-membership proofs	SATISFIED (260-byte content-binding proofs; E1)
4	Explicit update proofs	SATISFIED (proof-based == tree-based; E2)
5	AUTH-READ mechanized	SATISFIED (PROVED BY K)
6	AUTH-UPDATE mechanized	SATISFIED (PROVED BY K)
7	AUTH-COMMIT / root-lineage binding mechanized	SATISFIED (PROVED BY K, RL set)
8	Gate soundness mechanized with anti-vacuity	SATISFIED branch-wise + witness; GA1 single-form BLOCKED and disclosed (content carried)
9	Authenticated composition	SATISFIED (per-step K + two-layer demos + Python chain)
10	Execution-binding boundary demonstrated and bounded	SATISFIED (KEVM; boundary statements locked)
11	Countermodels CM-E1..E9 demonstrated rejected	SATISFIED (permanent suites)
12	Cross-layer Python/K/KEVM byte consistency	SATISFIED (18/18)
13	No weakening of Phase 1D-R1; zero regression	SATISFIED (baseline re-verified at start; frozen diff empty; permanent suites green at close)
14	Closure packaging: report, bundle, provenance, git	SATISFIED by this closure (bundle + PROVENANCE + phase1e branch push)

Phase 1F remains out of scope and is not entered by this closure.

11. Artifact index

Artifact	Path
Model spec (FROZEN)	phase1e/semantics/MODEL.md
K semantics (additive)	phase1e/semantics/srw3auth.k, demo srw3auth-demo.k
K proofs + negative controls	phase1e/proofs/{authenticated_read,authenticated_update,root_lineage_binding,gate_auth_commit}.k, proofs/negative_controls/
KEVM binding + demos	k/kevm/srw3-auth-evm.k, k/kevm/demos/evm_auth_{commit,stale}.srw3evm
Python reference + suites	phase1e/python/{auth_model,auth_verify,test_auth_verify,auth_mutations}.py
Transcripts (Parts 0-6)	phase1e/transcripts/ and transcripts/audit/phase1e/
Cross-layer script	scripts/phase1e_crosslayer.py, claim splitter scripts/split_auth_claims.py
Provenance	phase1e/provenance/ (starting state, hashes, reproducibility)